7-зертханалық жұмыс
Дашбордтар және алерттер

2-бөлім: 5-7-тармақтар және бизнес-метрика қосымшасы

Мәлімет	Мәні
Орындаған	Есімханова Аружан Түсіпханқызы
Тобы	25-ИСТК-2о
Университет	ВКТУ
Күні	2026 жылғы 24 қазан
Тапсырма	14-материал; 7-зертханалық жұмыс, 2-бөлім

Мақсаты

Дашбордты Git-тегі конфигурация ретінде дамыту, нақты ақаулар арқылы алерттердің pending және firing күйлерін өлшеу. HTTP сұрауларының метрикасын сәтті сақталған тапсырмалардың бизнес-санауышымен салыстыру.

Нәтиже

«Tasks created per minute» және бизнес-метрика панельдері қосылды; UID demo-api сақталды. a-е бес сценарий орындалды: 500 қателері, баяу жауап, процесс ақауы, жадтың таусылуы және API қолжетімсіздігі. Алты қажетті firing ауысуы бастапқы алерт күйлерінің журналымен расталды.

Жад тәжірибесінің бірінші әрекетінде OOM ескертуден ерте болды; бұл нәтиже жасырылмай сақталды. Қайталанған d сценарийінде алдымен ContainerHighMemory firing, кейін OOM (exit code 137) және ContainerRestarted алынды. API қалпына келді, PostgreSQL деректері сақталды.

PR #23 · Бастапқы дәлелдер · Қайта орындау нұсқаулығы

Тәжірибе нұсқасы - lab14-7d78755. Кестелердегі уақыт UTC: жергілікті уақытқа 5 сағат қосылады. Есеп тек нақты алынған нәтижелерді сипаттайды; соңғы main deploy бөлек тексеру файлына жазылады.

1. Дашбордты код ретінде сақтау

Файл: monitoring/grafana/dashboards/demo-api.json. Дашборд UID - demo-api, бума - DevOps course. Бастапқы бес панель сақталды; id=6 және id=7 жаңа панельдері қосылды.

Tasks created per minute

```
sum(increase(http_requests_received_total{
  job="demo-api", controller="Tasks",
  action="Create", code="201"}[1m]))
```

Соңғы минутта HTTP 201 жауабын берген Tasks/Create сұраулары есептеледі. increase өлшем терезесінің шеттерін экстраполяциялайды, сондықтан нәтиже бөлшек болуы мүмкін. Панельдің өлшемі - тапсырма/мин.

Business tasks created per minute

```
sum(increase(demo_tasks_created_total{job="demo-api"}[1m]))
```

Екінші панель HTTP жауаптарын емес, TaskService арқылы сәтті сақталған тапсырмаларды санайды. Екі панельдің дереккөзі - Prometheus, UID prometheus. Екі сұрау да Git ішінде болғандықтан, Grafana қайта жасалғанда да қалпына келеді.

Docker Desktop-тағы нақты түзету

Файл контейнер ішінде жаңарды, бірақ Grafana-дағы панель саны 5 болып қалды. Себебі bind mount арқылы файл өзгерісі туралы watch оқиғасы жетпеді. Provisioning ішіне updateIntervalSeconds:20 қосылып, тек Grafana қайта іске қосылды.

```
providers:
  - name: default
    folder: DevOps course
    type: file
    updateIntervalSeconds: 20
    options:
      path: /var/lib/grafana/dashboards
```

Нәтиже: 7 панель жүктелді. Кейін description өзгертіліп, 19,429 секундтан соң екінші restart жасамай API арқылы тексерілді. Файлдағы панель ID/title/PromQL жүктелген спецификациямен дәл сәйкес.

Орындау тәсілінің шегі

Work браузерді басқаруға рұқсат бермеді. Сондықтан панель Classic JSON файлына тікелей енгізіліп, нақты Grafana API арқылы тексерілді. UI ішіндегі Add/Export/Copy to clipboard қадамдары орындалды деп көрсетілмейді; интерфейс суреттері ойдан жасалған жоқ.

Дашборд JSON · Нақты polling тексеруі · Grafana жауабы

2. Барлық сценарийдің уақыт кестесі

Күйлер Prometheus /api/v1/rules?type=alert адресінен шамамен әр секунд сайын оқылды. Кестеде алғашқы pending және firing бақылаулары берілген. Ереженің lastEvaluation және activeAt уақыттары толық JSON ішінде сақталған. Бақылау дәлдігі - шамамен бір poll аралығы; subsecond цифрлары дәл абсолют шекараға кепіл болмайды.

Сценарий / алерт	Басталуы UTC	pending байқалды	firing байқалды	Басталудан firing-ке, с
a-errors ApiHighErrorRate	17:14:53.813	17:15:33.861	17:16:03.627	69,814
b-slow ApiSlowRequests	17:16:33.694	17:17:23.571	17:17:53.189	79,495
c-crash ContainerRestarted	17:18:08.846	-	17:18:24.770	15,925
d-memory ContainerHighMemory	17:28:35.433	17:28:55.460	17:29:09.804	34,372
d-memory ContainerRestarted	17:28:35.433	-	17:29:40.476	65,043
e-down ApiDown	17:29:56.101	17:30:08.123	17:30:38.877	42,776

ContainerRestarted ережесінде for жоқ: бірінші true evaluation кезінде бірден firing болады. Сондықтан c және d/restart жолдарындағы pending ұяшығы «-»; жасанды уақыт қойылған жоқ.

Сценарийлердің оқшауланууы

a аяқталған соң ApiHighErrorRate inactive болғаны, b аяқталған соң ApiSlowRequests inactive болғаны күтілді. c алдында бастапқы deploy-дан қалған ContainerRestarted сөнді. c мен d арасында толық бес минуттық терезе бітіп, ереже inactive-ке қайтқанша күтілді. Жадтың ерте OOM әрекетінен кейін де жаңа таза терезе күтілді.

a-c - бірінші іске қосудың өзгертілмеген сәтті тәжірибелері; d-e - бөлек жалғастыруда орындалды. Екі журнал бір хронологияда сақталды. Алғашқы сәтсіз d өлшемі d-initial-premature-oom деп жеке белгіленді; кестеге сәтті d сценарийінің уақыттары енгізілді.

Уақыттар, activeAt және lastEvaluation · Толық бастапқы күй журналы

3. a-b: қателер және баяу жауаптар

a. ApiHighErrorRate

```
1..60 | ForEach-Object {
  curl.exe -s -o NUL http://localhost:8088/api/chaos/error
  Start-Sleep 1
}
```

60 нақты сұраудың бәрі HTTP 500 берді. Процесс жұмысын жалғастырды: before/after ID және RestartCount бірдей. Ереже 5xx үлесі 5%-дан асқанда pending болды; for=30s орындалған соң firing-ке өтті. RPS панелінде 500 қатары мәселені көрсетеді.

Сақталған API журналында Error, InvalidOperationException және «Chaos: искусственная ошибка» бар. Осы іске қосуда UI автоматты түрде қаралмады; docker logs шығысы бастапқы тілінде сақталды.

b. ApiSlowRequests

```
1..40 | ForEach-Object {
  curl.exe -s -o NUL "http://localhost:8088/api/chaos/slow?ms=1500"
}
```

40 сұрау HTTP 200 берді; ең аз өлшенген жауап 1,501 с. Жауап дұрыс болғанымен, жалпы p95 >1 с шарты for=30s бойы сақталып, алерт firing болды. Latency p95 панелі бұрын ps/logs көрсете алмаған пайдаланушы күтуін анықтайды.

Маңызды айырма

ApiHighErrorRate - сервер қателерінің үлесін, ApiSlowRequests - уақыт гистограммасының 95-процентилін бақылайды. Баяу сұрау міндетті түрде 500 бермейді және application Error жазбасын жасамайды. p95 үшін баяу сұраулардың үлесі жеткілікті болуы керек; бір жалғыз баяу сұрау жалпы p95-ті өзгерте бермейді.

Қалпына келу

Қате және баяу сұрау циклдері тоқтаған соң жаман үлгілер минуттық терезеден бірден шықпайды. Сондықтан алерттер де бірден сөнбейді. Екі ереже inactive болғаннан кейін ғана келесі тәжірибеге көшілді.

a: HTTP жауаптары және firing кесіндісі · a: Error журналы · b: өлшенген кідірістер

4. c: процестің авариялық аяқталуы

```
curl.exe -X POST http://localhost:8088/api/chaos/crash
```

Белгі	Нәтиже
HTTP жауабы	202 Accepted; одан кейін процесс аяқталады
Docker оқиғалары	die(exitCode=1) → start → healthy
Контейнер ID	Сақталды
RestartCount	0 → 1
ContainerRestarted	firing; for жоқ
ApiDown	Күй журналында inactive; firing жоқ

Журнал мен метрика

Application журналы Critical деңгейінде авариялық аяқталуды, кейін жаңа процестің startup хабарларын көрсетеді. cAdvisor container_start_time_seconds мәнінің өзгеруін жазады; ереже max by(name) арқылы қатарларды біріктіріп, соңғы бес минуттағы changes мәнін есептейді.

```
changes((max by (name) (
  container_start_time_seconds{name="devops-labs-api-1"}
))[5m:5s]) > 0
```

Неге бірден pending жоқ?

Restart - болған оқиға. for параметрі берілмеген, сондықтан ереже оны бірінші evaluation кезінде firing деп белгілейді. Бес минуттық терезе оқиғаны біршама уақыт көрінетін етеді. Жаңа startup мәні терезеден ескі мән толық шыққанша өзгеріс ретінде сақталады.

d-ге дейін күту

Бұл firing кезеңі келесі OOM-мен шатаспау үшін ереже inactive-ке оралғанша күтілді. Қауіпсіздікті әлсірету, for уақытын қысқарту немесе алертті қолмен өшіру қолданылған жоқ.

c: before/after және уақыттар · c: Critical/startup журналы · c және алғашқы OOM оқиғалары

5. d: жад ескертуі және OOM

Алғашқы әрекет неге қайталанды?

Әдістемедегі 80+100 MiB бөлінгенде, жүктемеден қалыптасқан бастапқы жад қосылып, процесс ContainerHighMemory үшін for=15s өтпей OOM арқылы аяқталды. Docker-де oom және die (exit code 137) бар, ал ескерту firing болмады. Бұл сәтсіз әрекет жеке дәлел ретінде толық сақталды.

Қайталанған тәжірибе

Алдыңғы ContainerRestarted inactive болғаннан кейін 80 MiB, кейін 20 MiB қадамдар бөлінді. Әр қадамнан кейін docker stats өлшенді; жад кемінде 84%-ға жеткенде бөлу тоқтатылды. Ескерту нақты firing болғанша күтіліп, содан кейін қосымша 60 MiB арқылы OOM жасалды.

Өлшем	Нақты нәтиже
Бөлінген порциялар, MiB	80 + 20 + 20 + 20 + 60
OOM алдындағы соңғы stats	88,63%
ContainerHighMemory	pending → firing, for=15s
Оқиға тізбегі	oom → die (exit code 137) → start → healthy
RestartCount	2 → 3
Контейнер және БД	API ID сақталды; PostgreSQL тоқтатылған жоқ

Панель мен журнал

Working set / limit панелі жадтың сатылы өсуін, 256 MiB шегін және restart-тен кейін төмендеуін көрсетеді. OOM кезінде application соңғы Error жазбасын қалдыруға үлгермеуі мүмкін; себептің шешуші дәлелі - Docker oom/die (exit code 137). Кейін startup журналы жаңа процестің басталғанын растайды.

Бұл сценарийде ешбір шек өзгертілген жоқ: API 256 MiB, RAM+swap 256 MiB, ескерту 80% / 15 с. Өзгергені - ақауға дейінгі жадты ескеретін бөлу порциялары. Бастапқы әрекетті сәтті ескерту тәжірибесі ретінде көрсетуге болмайды.

d: өлшемдер, порциялар және firing · Сәтті d-е Docker оқиғалары · Алғашқы аяқталмаған d әрекеті

6. e: қолжетімсіздік және қалпына келу

```
docker compose stop api
# ApiDown pending, кейін firing болғанша күту
docker compose start api
```

Бұл жолы API әдейі тоқтатылып, /metrics қолжетімсіз күйінде 30 с-тан ұзақ ұсталды. Prometheus up=0 арқылы ApiDown pending және firing күйлерін тіркеді. API up панелі 0 көрсетеді; тоқтау кезінде жаңа application сұрау логтары болмайды.

Кезең	UTC уақыты немесе нәтиже
stop аяқталды	17:29:56.579
start аяқталды	17:30:55.003
Соңғы Docker health	healthy
/health/live	200
/health/ready	200
ApiDown	API қайта басталғаннан кейін inactive

Деректердің сақталуы

10-12-материалдардың үш бақылау жобасы мен тапсырмасы бастапқы объектілерімен дәл салыстырылды. 13-материалдың 1200 InProgress тапсырмасы сақталды. Осы жұмыстың бизнес-санауыш тексеру объектілері және екі жүктеме жобасының тапсырмалары да PostgreSQL-де қалды.

Жүктеме мен диагностика

Екі іске қосуда фондық load.ps1 қолданылды. Crash/stop/OOM кезінде байланыс үзілуі күтіледі; бұл жолдар load-console ішінде сақталған және сәтті сұрау ретінде саналмайды. Тәжірибе біткенде тек осы скрипт іске қосқан жүктеме процесі тоқтатылды. Өзге жобаның PostgreSQL контейнері өзгертілген жоқ.

Нәтижені түсіндіру

unless-stopped қолмен stop жасалған контейнерді өздігінен қайта көтермейді. Оның healthy күйге қайтуы - нақты start пәрменінің нәтижесі. Ал c және d кезінде процестің авариялық аяқталуы restart саясаты арқылы қалпына келді.

e: күй және уақыттар · Тәжірибеден кейінгі API/targets · Сақталған бақылау деректері

7. Тапсырудағы үш сұраққа жауап

1. Ақау басталғаннан firing-ке дейін неге уақыт өтеді?

Prometheus алдымен келесі scrape кезінде жаңа метриканы алуы керек: стендте аралығы 5 с. rate/increase және p95 минуттық терезеге сүйенеді, сондықтан қателер не баяу сұраулар жеткілікті үлеске жетуі тиіс. Кейін келесі evaluation күтіледі: оның аралығы да 5 с. Шарт true болғанда for=30s немесе for=15s таймері басталады.

Бұл әрқашан «5+60+5+30» деген тұрақты қосынды емес. Минуттық терезе алдын ала толы болуы мүмкін; шекті үлес 60 с аяқталмай-ақ жиналады. for кезінде шарт жоғалса, таймер тасталады. Бақылаудың бір секундтық poll аралығы да көрген уақытқа қосымша айырма береді. Кестеде осы нақты тәжірибенің уақыттары келтірілген.

2. c сценарийінде ApiDown неге firing болмады?

Процесс die (exit code 1) арқылы аяқталды, бірақ unless-stopped оны тез қайта бастады. ApiDown шарты up=0 күйін 30 с бойы үздіксіз талап етеді. Үзіліс бұдан қысқа болғандықтан, ұзақ қолжетімсіздік расталмады. Осы тәжірибенің күй журналында c кезінде ApiDown үнемі inactive болды; өте қысқа үзілісті scrape мүлде байқамауы да мүмкін. Restart фактісін ContainerRestarted ережесі for-сіз ұстап алды.

3. 6-зертханалық жұмыстың 9-тармағындағы қай мәселе енді көрінеді?

Баяу, бірақ HTTP 200 қайтаратын сұрау. Бұрын контейнер healthy, процесс тірі, application Error жоқ болып, пайдаланушының ұзақ күтуі ps пен error log арқылы көрінбеді. Енді http_request_duration_seconds гистограммасы мен Latency p95 панелі кешігуді көрсетеді; p95 > 1 с / 30 с шарты ApiSlowRequests алертін іске қосады. controller/action бойынша p95 сұрауы Chaos/Slow әрекетін бөлек табуға мүмкіндік береді.

Код, контейнер ресурсы және пайдаланушы симптомы әртүрлі дерек береді: HTTP 500 - сұрау қатесі, die1 - процесс аяқталуы, OOM (exit code 137) - жад шегінен асып өлу, up=0 - метриканы ала алмау. Диагноз үшін оларды бір оқиға деп араластыруға болмайды.

8. Қосымша: бизнес-санауыш

```
private static readonly Counter TasksCreated = Metrics.CreateCounter(
  "demo_tasks_created_total",
  "Successfully persisted tasks created by TaskService.");

var task = await tasks.CreateAsync(projectId, title, description, dueDate, ct);
TasksCreated.Inc();
```

TaskService ішіндегі Inc репозиторий тапсырманы сәтті сақтағаннан кейін ғана орындалады. Егер жоба жоқ болса, сервис оған жетпей null қайтарады. Request validation қатесі контроллерге дейін кері қайтарылады. Сәтсіз persistence кезінде де Inc орындалмайды. Жаңа тәуелділік қосылмады: prometheus-net бұрыннан бар.

Нақты тексеру	Санауыш
Бастапқы мән	503.0
Бір 201 сәтті құрудан кейін	504.0
400 және 404 жауаптарынан кейін	504.0
Процесс қайта басталса	Жаңа процестің санауышы нөлден басталады; БД тапсырмалары сақталады

HTTP 201 санауышымен қашан айырма болады?

Қалыпты HTTP жолында екі өсім жақын болуы тиіс. Бизнес-санауыш тапсырма сақталған сәтте өседі, HTTP метрикасы жауап аяқталған кезде жазылады. Сақтаудан кейін жауапты жазу/сериализациялау сәтсіз болса немесе клиент байланысын үзсе, бизнес оқиғасы болды, бірақ сәтті 201 жауап клиентке жетпеуі мүмкін. Кейін HTTP-ден тыс background job осы сервисті шақырса да бизнес-санауыш өседі.

Бұл ерекше ақау осы жұмыста әдейі жасалған жоқ; ол семантикалық айырманы түсіндіреді. Екі counter да process lifetime-ға тиесілі; олар БД-дағы барлық тарихи тапсырманың санын білдірмейді. Өсімді rate/increase арқылы оқу restart reset-ін ескереді. TaskId/email сияқты жоғары кардиналды лейблдер қосылмады.

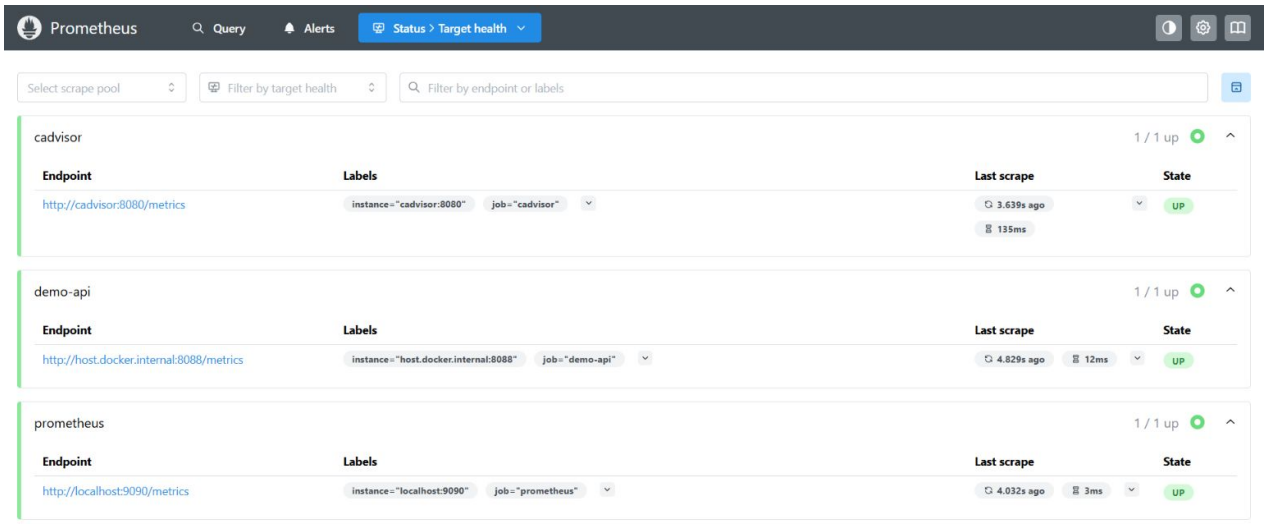
Тест және Postman

Жаңа интеграциялық тест нақты /metrics мәнінің өзгерісін тексерді: бір сәтті сақтауда +1, 400/404 кезінде өзгеріс жоқ. Толық InMemory 40/40. Жүктеме тоқтаған соң нақты Postman 10 сұрау / 17 тексеру, қате 0, exit code 0. Коллекция бұлтқа жарияланған жоқ.

TaskService коды · Санауыш тесті · Бизнес өсімінің нақты қатары · HTTP 201 өсімінің нақты қатары

9. Жинақталған дәлелдер және шектеулер

Бірінші бөлімнен сақталған Target health



1-сурет. 13-материалдағы бастапқы шынайы Chrome суреті: 3 UP. Бұл 14-материалдың жаңа скриншоты деп көрсетілмейді.

13-материалдағы төрт жеке PromQL сұрауы, графиктері және түсіндірмелері сол бөлімнің PDF-інде және evidence/lab07-metrics каталогында сақталды. Осы бөлімнің соңғы targets JSON-ы қалпына келген 3 UP күйін жаңадан растайды.

Қайта орындау

```
docker compose -f monitoring/compose.yaml up -d
python scripts/verify-lab14.py --tag lab14-<functional-sha> --output <new-directory>
# Жүктеме тоқтап, API қалпына келгеннен кейін:
postman collection run postman/lab07-alerts.postman_collection.json --no-report-events
```

Скрипт тек devops-labs-api-1 және нақты lab14 жергілікті тегін қабылдайды. API 256 MiB және swap 256 MiB шегін тексереді. Ол әдейі crash/OOM/stop жасайды; deploy-мен қатар іске қосуға болмайды. finally API-ді қайта іске қосып, өзі ашқан бақылау/жүктеме процестерін жабады.

Жариялау және адал есеп беру

PR #23 міндетті PostgreSQL test арқылы тексеріледі, содан кейін main CI/CD іске қосылады. Техникалық ревью бір иенің екі аккаунтымен жасалады; бұл тәуелсіз жұптық ревью емес. Local approval ассистент тарапынан пайдаланушының тапсырмасы бойынша орындалады. Соңғы main/runtime, файл хештері және деректердің сақталуы lab14-final-verification.json релиз asset-інде тіркеледі.

Браузер рұқсаты болмағандықтан UI экспорт және a-e кезінде Grafana/Dozzle-ны автоматты визуалды қарау орындалмады. Олардың орнына жүктелген Grafana спецификациясы, Prometheus жауаптары және контейнер журналдары берілді. Алғашқы OOM, жүктеменің күтілетін transport қателері және сәтті жалғастыру бір-бірінен нақты ажыратылды.

PR #23 · Postman нәтижесі · Релиз және соңғы тексеру

Prometheus alerting rules · Grafana provisioning / polling